

Introduction à TypeScript

Qu'est-ce que TypeScript ?

TypeScript est un langage de programmation open-source développé par Microsoft. Il s'agit d'un sur-ensemble typé de JavaScript qui ajoute des fonctionnalités de typage statique et de vérification de type au langage JavaScript. TypeScript permet aux développeurs de détecter les erreurs de type lors de la compilation, ce qui peut améliorer la qualité du code et faciliter la maintenance des applications.

TypeScript est conçu pour être compatible avec JavaScript, ce qui signifie que tout code JavaScript valide est également un code TypeScript valide. Les développeurs peuvent progressivement adopter TypeScript dans leurs projets JavaScript existants.

Avantages de TypeScript

1. **Typage statique** : TypeScript permet de définir des types pour les variables, les fonctions et les objets, ce qui aide à détecter les erreurs de type avant l'exécution du code.
2. **Meilleure lisibilité et maintenabilité** : Le typage explicite améliore la lisibilité du code et facilite la compréhension des intentions du développeur.
3. **Support des fonctionnalités modernes de JavaScript** : TypeScript prend en charge les dernières fonctionnalités de JavaScript, y compris les classes, les modules et les fonctions fléchées.
4. **Outils de développement améliorés** : TypeScript offre une meilleure autocomplétion, une navigation dans le code et une refactorisation grâce à son système de types.
5. **Interopérabilité avec JavaScript** : TypeScript peut être utilisé avec des bibliothèques JavaScript existantes, ce qui facilite son adoption progressive.

Désavantages de TypeScript

1. **Courbe d'apprentissage** : Les développeurs doivent apprendre les concepts de typage statique et les fonctionnalités spécifiques de TypeScript, ce qui peut représenter une courbe d'apprentissage.
2. **Temps de compilation** : TypeScript nécessite une étape de compilation pour convertir le code TypeScript en JavaScript, ce qui peut ralentir le processus de développement.
3. **Complexité accrue** : L'ajout de types et de fonctionnalités supplémentaires peut rendre le code plus complexe, ce qui peut être un inconvénient pour les petits projets ou les développeurs débutants.

Créer un nouveau projet TypeScript avec Vite

Pour créer un nouveau projet TypeScript avec Vite, suivez ces étapes :

1. **Créer un projet avec Vite, ensuite choisissez le modèle TypeScript**

```
npm create vite@latest chemin/vers/mon-projet
```

2. **Installer les dépendances** : Accédez au répertoire de votre projet et installez les dépendances :

```
cd mon-projet
npm install
```

Principes clés de TypeScript

Lorsque vous travaillez avec TypeScript, il est important de comprendre certains principes clés du langage. Vous devez être à l'aise avec les concepts suivants :

- La syntaxe de TypeScript et les différences par rapport à JavaScript
- Les types de base et les annotations de type
- Les interfaces et les types personnalisés

Syntaxe de TypeScript

TypeScript utilise une syntaxe similaire à celle de JavaScript, avec quelques ajouts pour le typage. Vous devez préciser les types des variables, des paramètres de fonction et des valeurs de retour. Voici un exemple simple :

```
function addition(a: number, b: number): number {
    let somme: number = a + b;
    return somme;
}
```

Types de base

TypeScript prend en charge plusieurs types de base, notamment :

- `number` : pour les nombres (entiers et flottants)
- `string` : pour les chaînes de caractères
- `boolean` : pour les valeurs true/false
- `any` : pour un type quelconque (à utiliser avec précaution)
- `void` : pour les fonctions qui ne retournent pas de valeur
- `number[]` : pour les tableaux de nombres (changer le type selon les besoins)
- `string[]` : pour les tableaux de chaînes de caractères (changer le type selon les besoins)
- `boolean[]` : pour les tableaux de valeurs booléennes (changer le type selon les besoins)
- `HTMLElement` : pour les éléments HTML dans le DOM. Spécifiez des types plus précis comme `HTMLDivElement`, `HTMLInputElement`, etc., selon le type d'élément HTML que vous manipulez.

Interfaces et types personnalisés

TypeScript permet de définir des interfaces et des types personnalisés pour structurer les données. Cela facilite la gestion des objets complexes. Voici un exemple d'interface :

```
interface Utilisateur {
    id: number;
    nom: string;
    email: string;
}
```

```
}  
function afficherUtilisateur(user: Utilisateur): void {  
    console.log(`ID: ${user.id}, Nom: ${user.nom}, Email: ${user.email}`);  
}
```

Compiler un projet TypeScript

Pour être exécuté dans un environnement JavaScript, le code TypeScript doit être compilé en JavaScript. Vous pouvez utiliser le compilateur TypeScript (**tsc**) pour cela. Lorsque vous créez un projet avec Vite, la configuration de compilation est généralement gérée automatiquement. Cependant, vous pouvez personnaliser les options de compilation en créant un fichier **tsconfig.json** à la racine de votre projet.

Le fichier **tsconfig.json** permet de définir les options de compilation, telles que la version cible de JavaScript, les répertoires d'entrée et de sortie, et d'autres paramètres.

Voici un exemple de fichier **tsconfig.json** :

```
{  
  "compilerOptions": {  
    "target": "es6",  
    "module": "esnext",  
    "strict": true,  
    "esModuleInterop": true,  
    "skipLibCheck": true,  
    "forceConsistentCasingInFileNames": true,  
    "outDir": "./dist"  
  },  
  "include": ["src/**/*"],  
  "exclude": ["node_modules", "**/*.spec.ts"]  
}
```

Après avoir configuré votre projet, vous pouvez compiler le code TypeScript en exécutant la commande suivante dans le terminal à la racine de votre projet. Cela générera les fichiers JavaScript dans le répertoire spécifié (par exemple, **./dist**).

```
npx tsc
```

Généralement, avec Vite, le processus de compilation est intégré dans le flux de développement, et vous n'avez pas besoin d'exécuter **tsc** manuellement. Vite s'occupe de la compilation à la volée lors du développement et de la construction du projet pour la production. Utilisez la commande suivante pour construire votre projet en production avec Vite.

```
npm run build
```

Quoi faire si je rencontre des erreurs de compilation ?

Si vous rencontrez des erreurs de compilation lors de l'utilisation de TypeScript, voici quelques étapes à suivre pour les résoudre :

1. **Lire attentivement les messages d'erreur** : Les messages d'erreur fournis par le compilateur TypeScript sont généralement très informatifs et peuvent vous indiquer exactement où se trouve le problème dans votre code.
2. **Vérifier les types** : Assurez-vous que les types que vous avez définis correspondent aux valeurs que vous essayez d'assigner ou de manipuler. Par exemple, si une variable est déclarée comme `number`, vous ne pouvez pas lui assigner une chaîne de caractères.
3. **Utiliser des assertions de type** : Si vous êtes certain qu'une valeur est d'un type spécifique, mais que TypeScript ne peut pas le déduire, vous pouvez utiliser des assertions de type pour informer le compilateur. Par exemple :

```
const valeur: any = "Hello, TypeScript!";
const longueur: number = (valeur as string).length;
console.log(longueur);
```

4. **Consulter la documentation** : La documentation officielle de TypeScript est une ressource précieuse pour comprendre les fonctionnalités du langage et résoudre les problèmes courants. Vous pouvez la consulter ici : <https://www.typescriptlang.org/docs/>

Convertir un projet de JavaScript en TypeScript

Parfois, vous pouvez avoir un projet JavaScript existant que vous souhaitez convertir en TypeScript. Voici quelques étapes pour effectuer cette conversion :

1. **Installez** globalement TypeScript si ce n'est pas déjà fait :

```
npm install -g typescript
```

2. **Créer** le fichier `tsconfig.json` : Si ce n'est pas déjà fait, créez un fichier `tsconfig.json` à la racine de votre projet pour configurer les options de compilation TypeScript. N'oubliez pas d'ajouter la source de votre code dans le champ `include`.

```
npx tsc --init
```

3. **Renommer** les fichiers : Changez l'extension des fichiers de `.js` à `.ts` pour indiquer qu'ils sont maintenant des fichiers TypeScript.
4. **Compiler** le projet : Exécutez le compilateur TypeScript pour vérifier les erreurs de type et compiler le code en JavaScript. Modifiez votre script de build pour inclure la compilation TypeScript dans le fichier `package.json`.

```
"scripts": {
  "build": "tsc && vite build"
```

```
}
```

5. **Tester** le projet : Après la compilation, testez votre projet pour vous assurer que tout fonctionne comme prévu. Corrigez les erreurs de type ou les problèmes qui pourraient survenir lors de l'exécution.

```
npm run build
```

Exemple de fichier `tsconfig.json` pour un projet converti de JavaScript à TypeScript :

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "esnext",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "outDir": "./dist"
  },
  "include": ["src/**/*", "assets/**/*"],
  "exclude": ["node_modules", "**/*.spec.ts"]
}
```

Ressources supplémentaires

- Documentation officielle de TypeScript : <https://www.typescriptlang.org/docs/>
- Guide de migration de JavaScript vers TypeScript :
<https://www.typescriptlang.org/docs/handbook/migrating-from-javascript.html>
- Guide pour débutants sur TypeScript : <https://www.freecodecamp.org/news/learn-typescript-beginners-guide/>